



香港中文大學(深圳)
The Chinese University of Hong Kong, Shenzhen

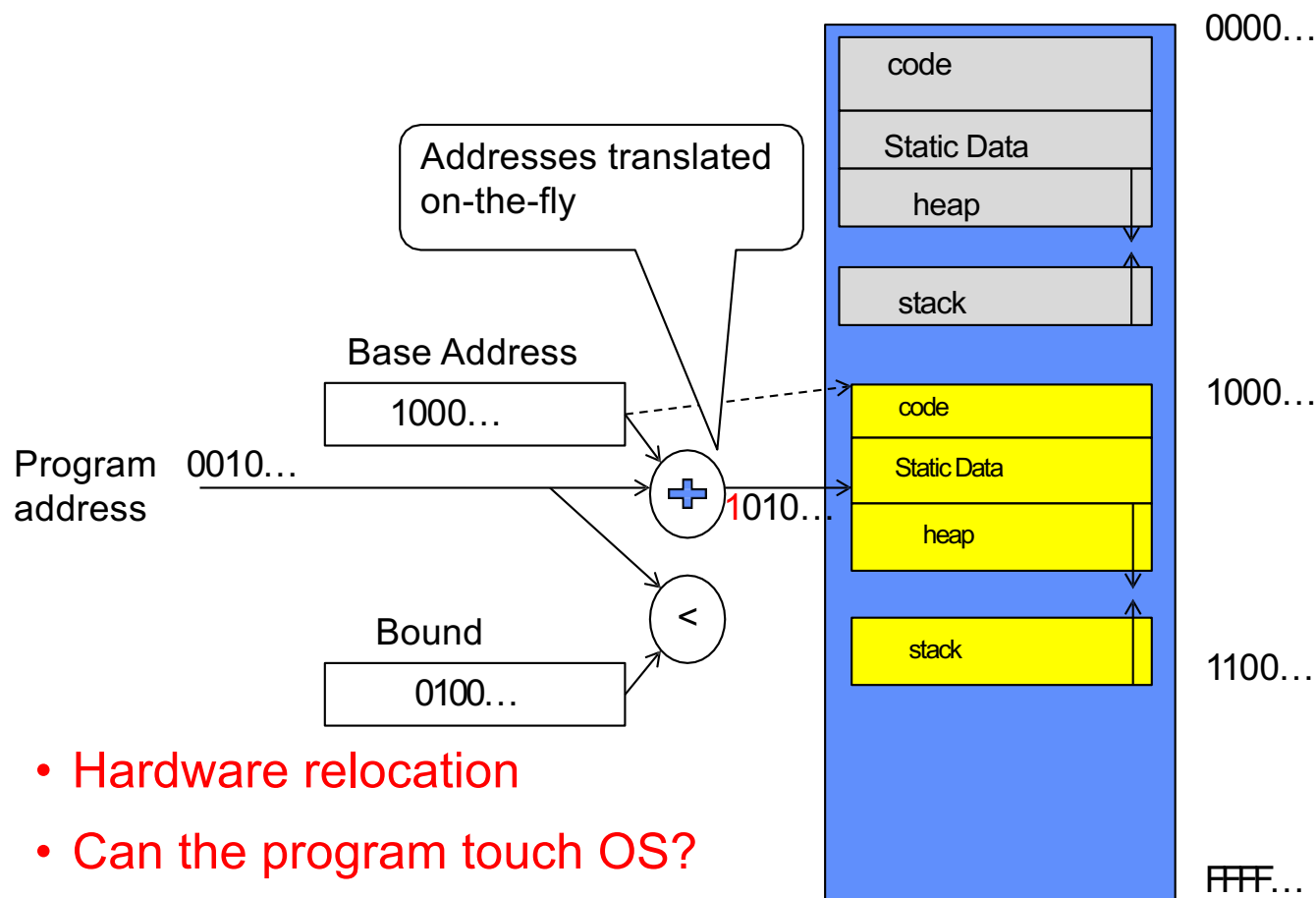
Operating Systems Lecture 16

Memory: virtual memory, page tables, caching

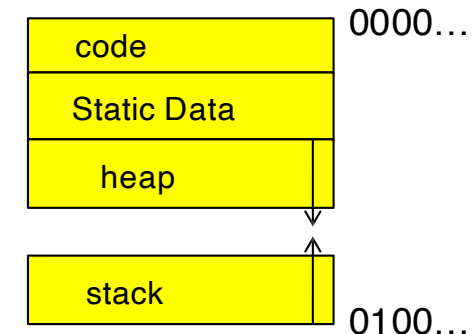
**Prof. Yunming XIAO
School of Data Science**

Acknowledgments: Ion Stoica and Matei Zaharia, Berkeley CS 162

Recall: base and bound (with translation)

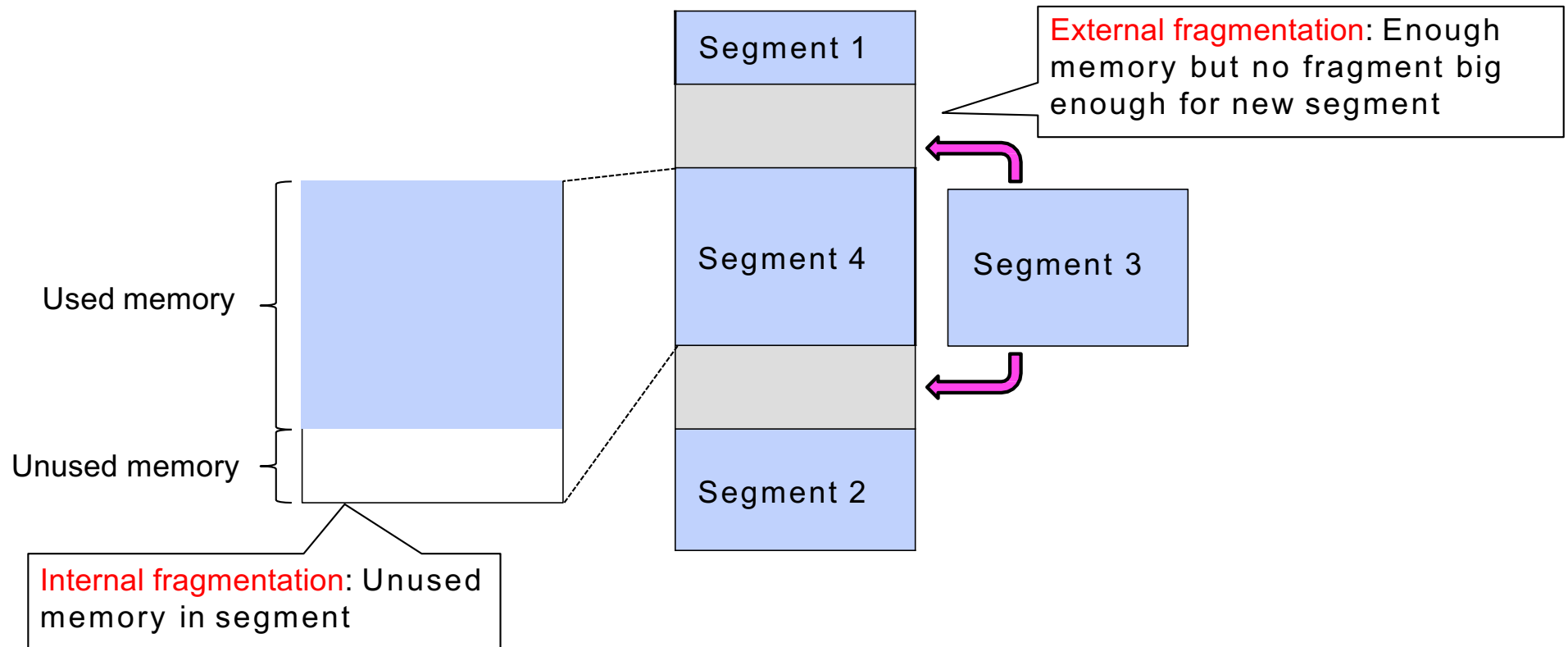


Original Program



- Hardware relocation
- Can the program touch OS?
- Can it touch other programs?

External vs. Internal fragmentation

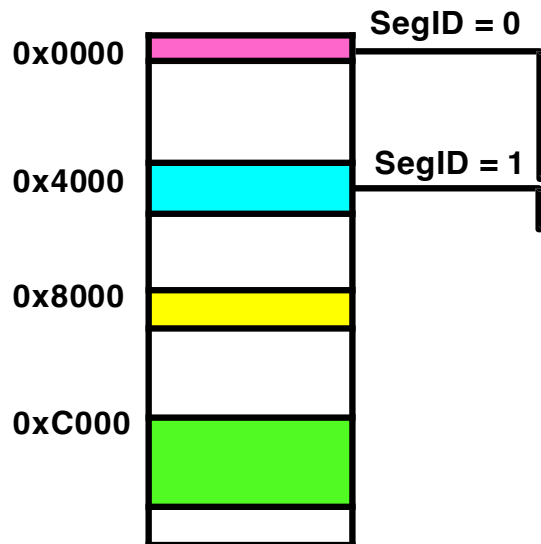


Recall: segmentation

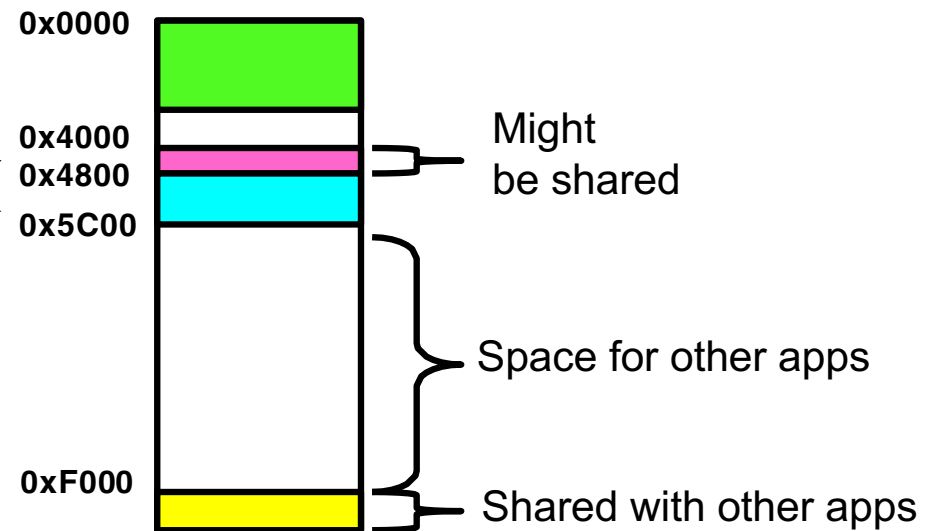


Virtual Address Format

SegID #	Base	Limit
0 (code)	0x4000	0x0800
1 (data)	0x4800	0x1400
2 (shared)	0xF000	0x1000
3 (stack)	0x0000	0x3000



Virtual Address Space

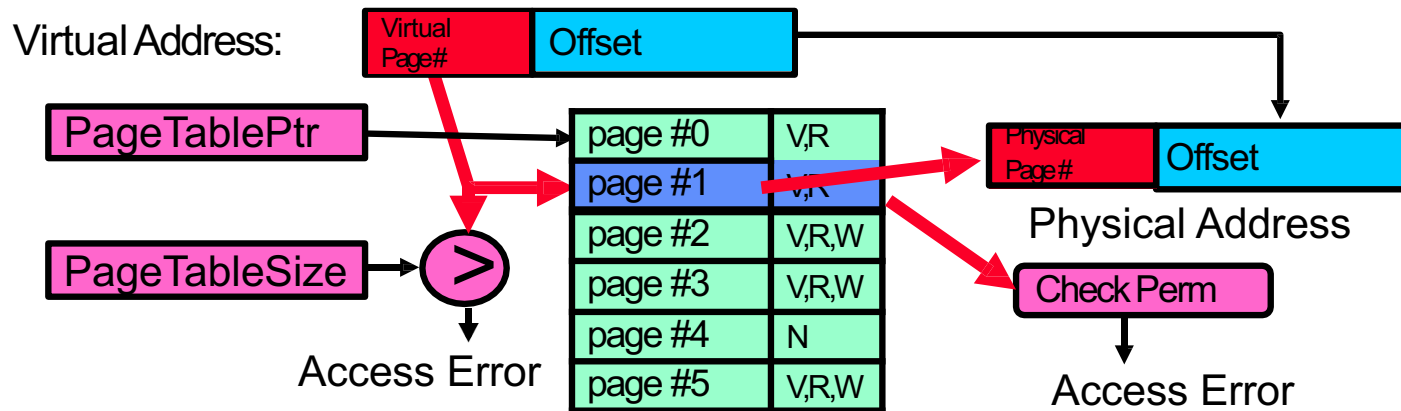


Physical Address Space

Recall: paging: physical memory in fixed size chunks

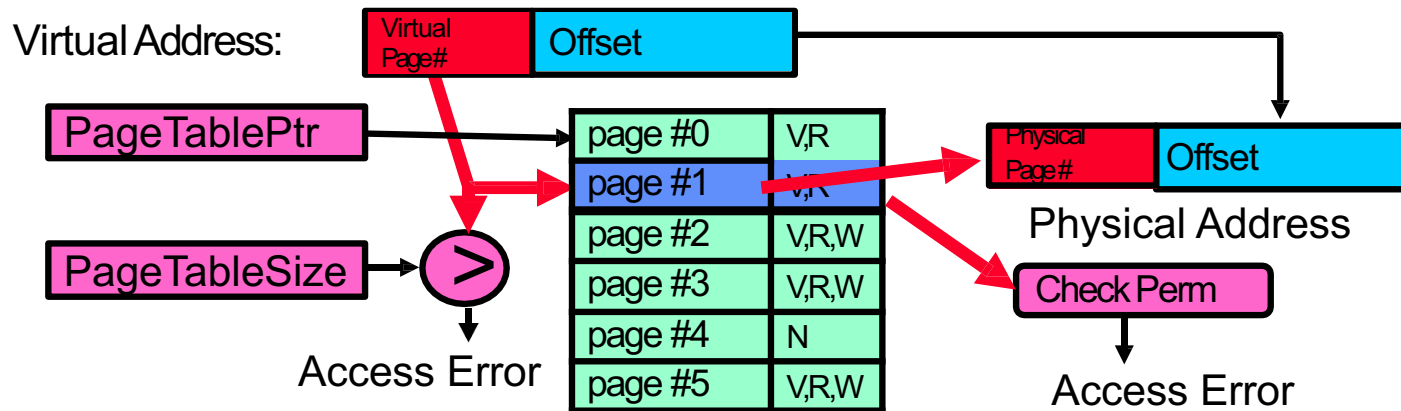
- Solution to fragmentation from segments?
 - Allocate physical memory in **fixed size** chunks (“**pages**”)
 - Every chunk of physical memory is equivalent
 - Can use simple vector of bits to handle allocation:
00110001110001101 ... 110010
 - Each bit represents page of physical memory
1 $\Rightarrow$ allocated, **0** $\Rightarrow$ free
- Should pages be as big as our previous segments?
 - No: Can lead to lots of internal fragmentation
 - Typically have small pages (1K-16K)
 - Consequently: need multiple pages/segment

Recall: how to implement simple paging?



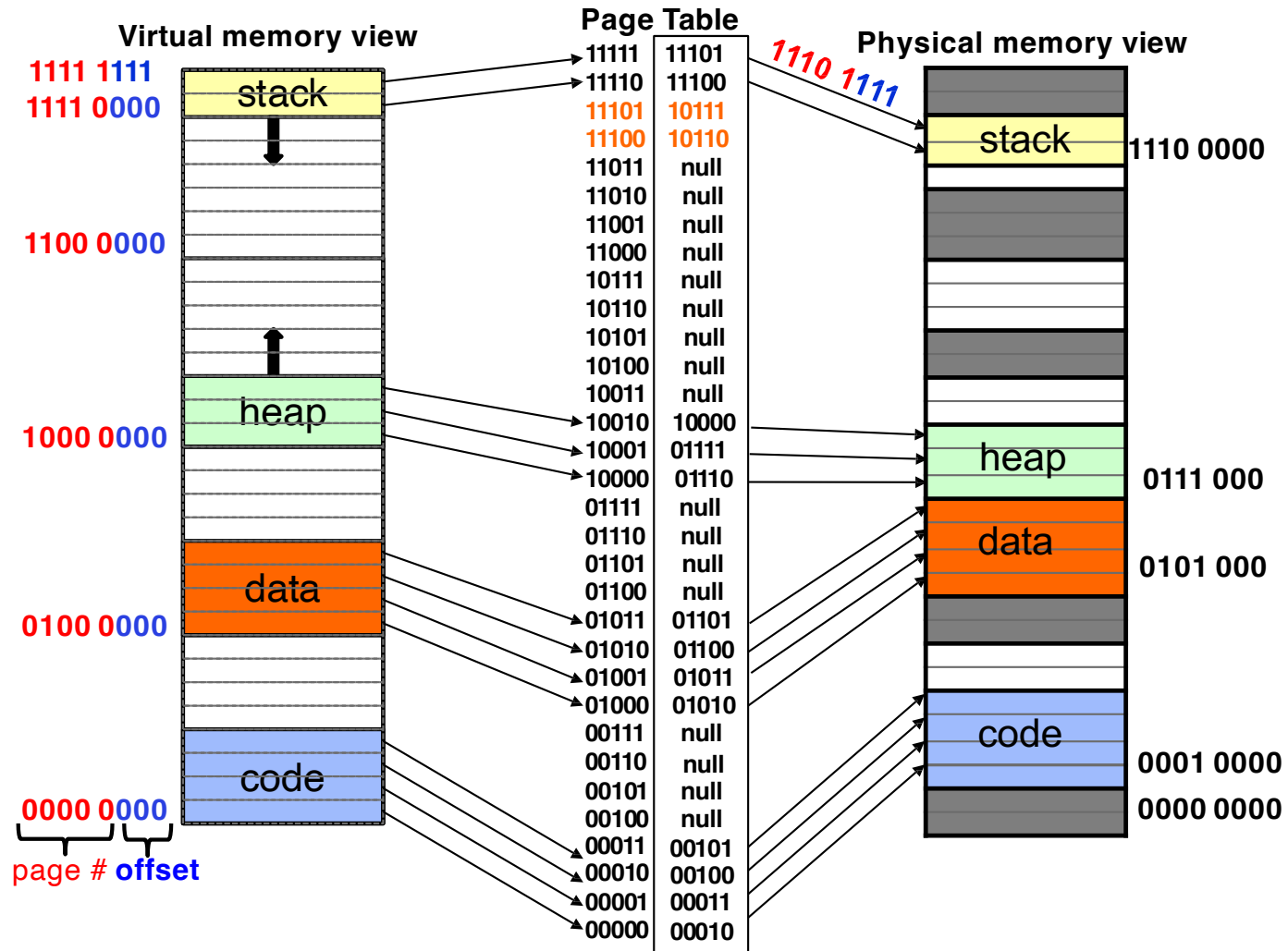
- Page Table (one per process)
 - Resides in physical memory
 - Contains physical page and permission for each virtual page (e.g. Valid bits, Read, Write, etc)

Recall: how to implement simple paging?

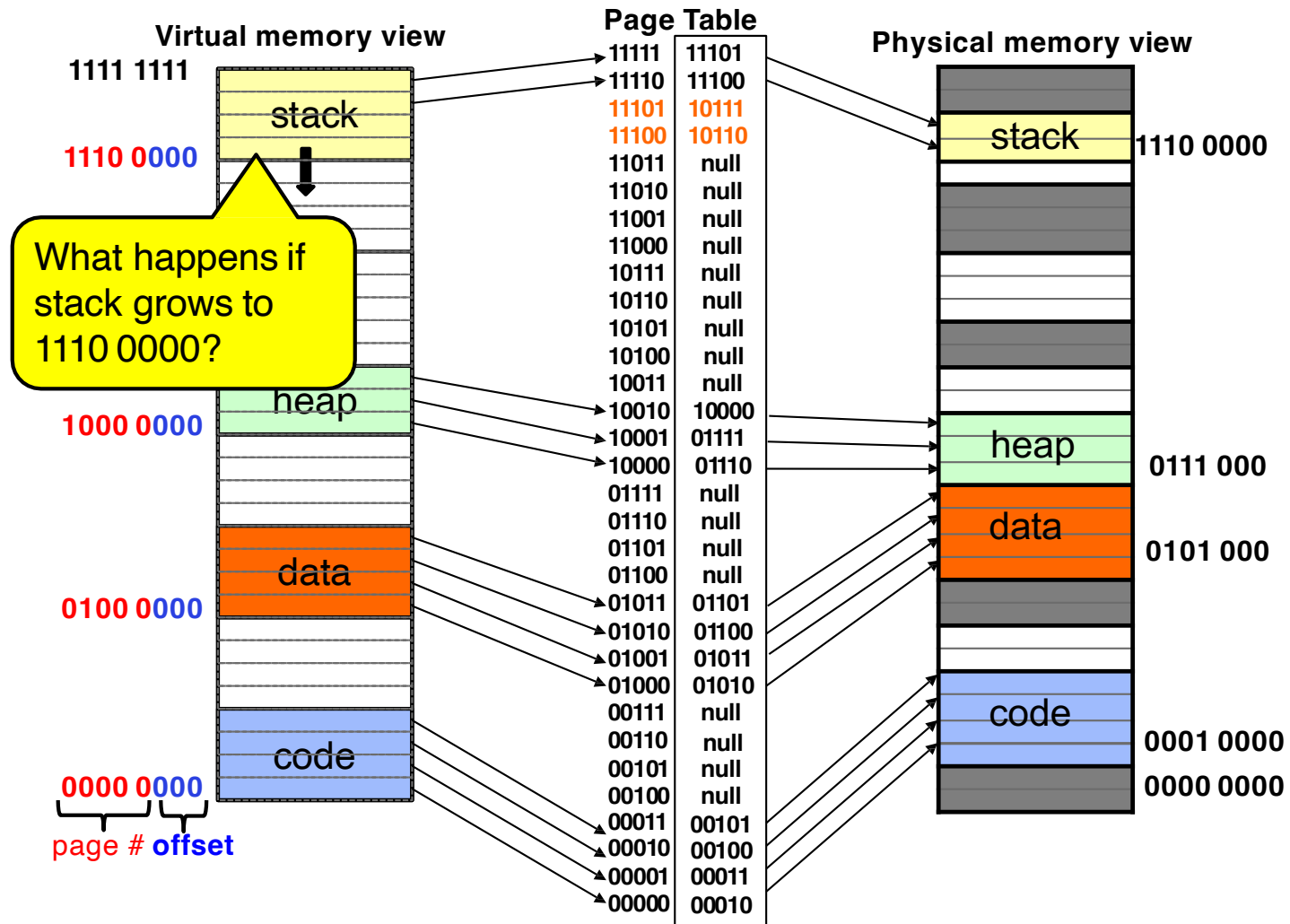


- Virtual address mapping
 - Offset from Virtual address copied to Physical Address
 - Example: 10 bit offset $\Rightarrow$ 1024-byte pages
 - Virtual page # is all remaining bits
 - Example for 32-bits: $32 - 10 = 22$ bits, i.e. 4 million entries
 - Physical page # copied from table into physical address
 - Check Page Table bounds and permissions

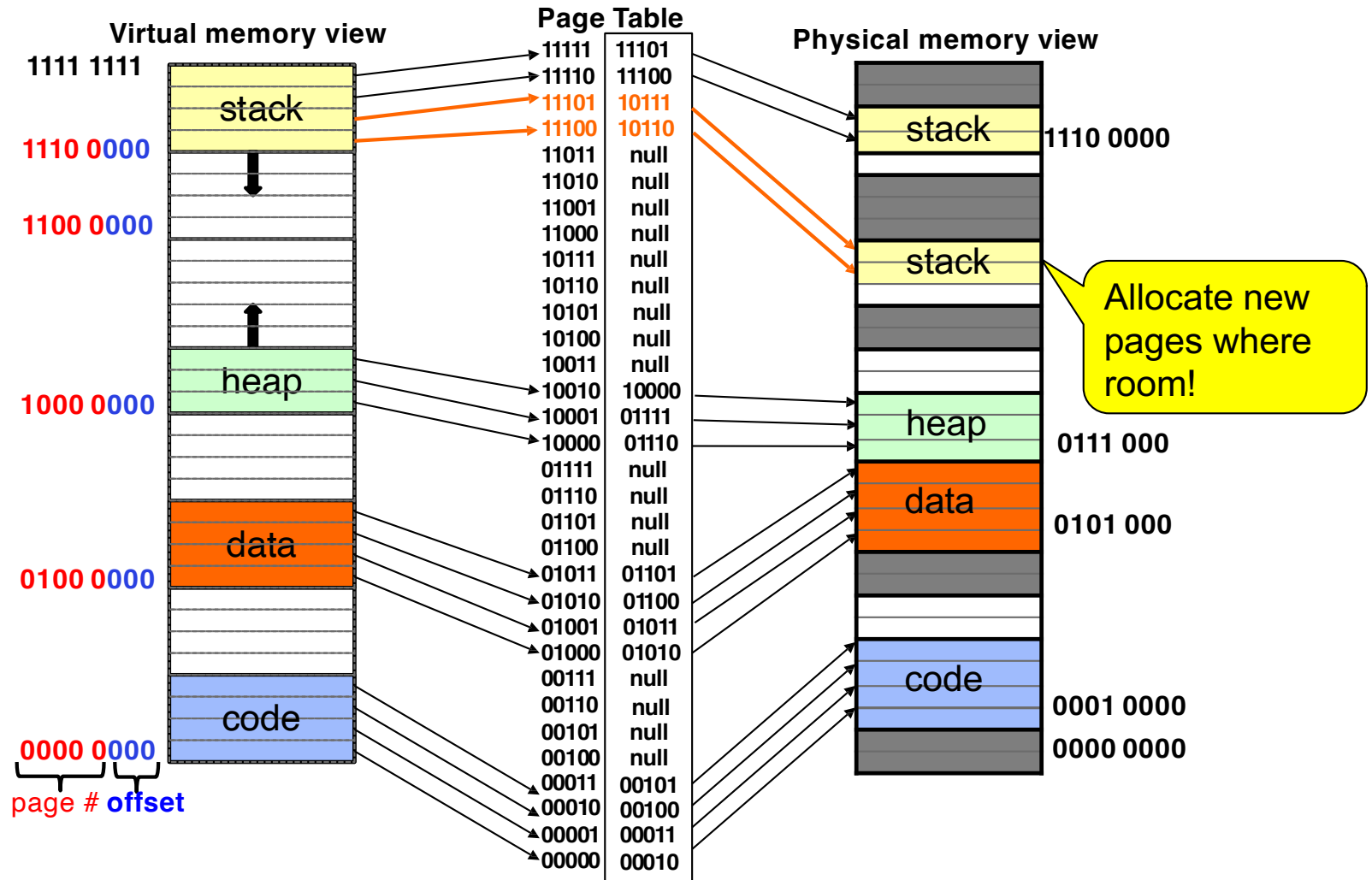
Recall: summary: paging



Recall: summary: paging



Recall: summary: paging



Page Table

Page Table discussion

- What needs to be switched on a context switch?
 - Page table pointer and limit
- What provides protection here?
 - Translation (per process) and dual-mode!
 - Can't let process alter its own page table!

Page Table discussion

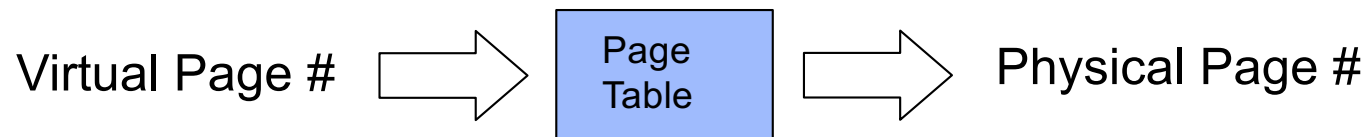
- Pros
 - Simple memory allocation
 - Easy to share
- Con: What if address space is sparse?
 - E.g., on UNIX, code starts at 0, stack starts at $(2^{31}-1)$
 - With 1K pages, need 2 million page table entries!
- Con: What if table really big?
 - Not all pages used all the time $\Rightarrow$ would be nice to have working set of page table in memory

Page Table discussion

- Simple Page table is way too big!
 - Does it all need to be in memory?
 - How about multi-level paging?
 - or combining paging and segmentation

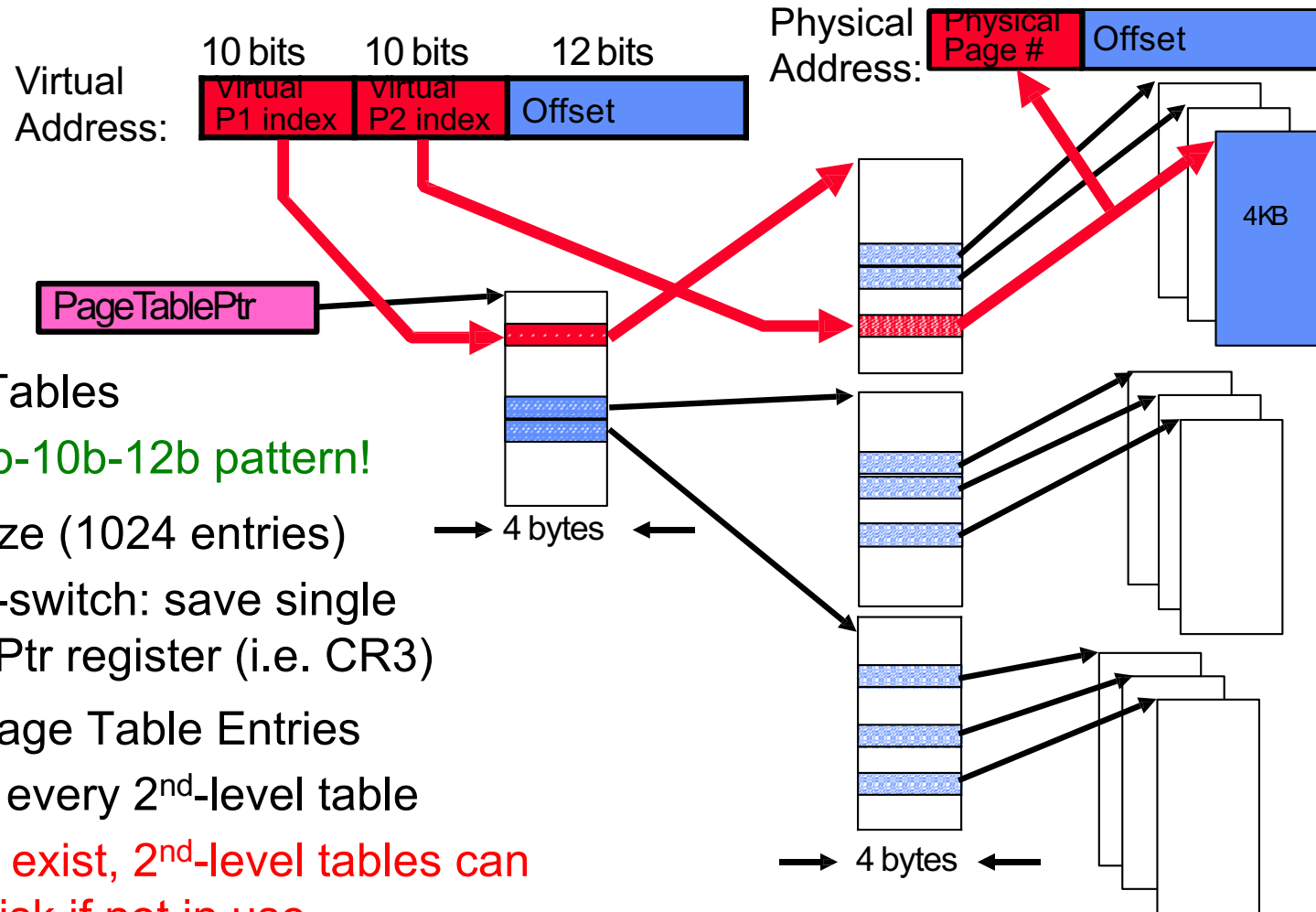
How to structure a Page Table

- Page Table is a map (function) from virtual page number (VPN) to physical page number (PPN)



- Simple page table corresponds to a very large lookup table
 - VPN is index into table, each entry contains PPN
- What other map structures can you think of?
 - Trees?
 - Hash Tables?

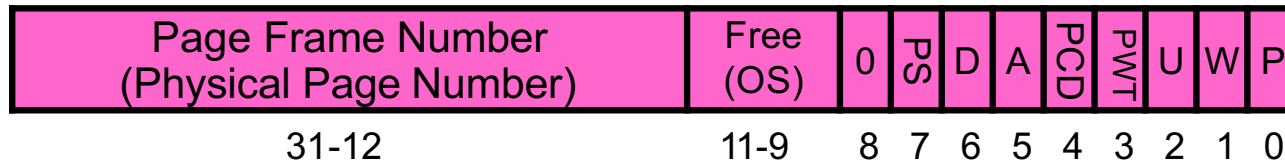
Fix for sparse address space: the two-level page table



- Tree of Page Tables
 - “Magic” 10b-10b-12b pattern!
- Tables fixed size (1024 entries)
 - On context-switch: save single PageTablePtr register (i.e. CR3)
- Valid bits on Page Table Entries
 - Don’t need every 2nd-level table
 - Even when exist, 2nd-level tables can reside on disk if not in use

What is in a Page Table Entry (PTE)?

- What is in a Page Table Entry (or PTE)?
 - Pointer to next-level page table or to actual page
 - Permission bits: valid, read-only, read-write, write-only
- Example: Intel x86 architecture PTE:
 - Address same format previous slide (10, 10, 12-bit offset)
 - Intermediate page tables called “Directories”



- P:** Present (same as “valid” bit in other architectures)
- W:** Writeable
- U:** User accessible
- PWT:** Page write transparent: external cache write-through
- PCD:** Page cache disabled (page cannot be cached)
- A:** Accessed: page has been accessed recently
- D:** Dirty (PTE only): page has been modified recently
- PS:** Page Size: PS=1⇒4MB page (directory only).
Bottom 22 bits of virtual address serve as offset

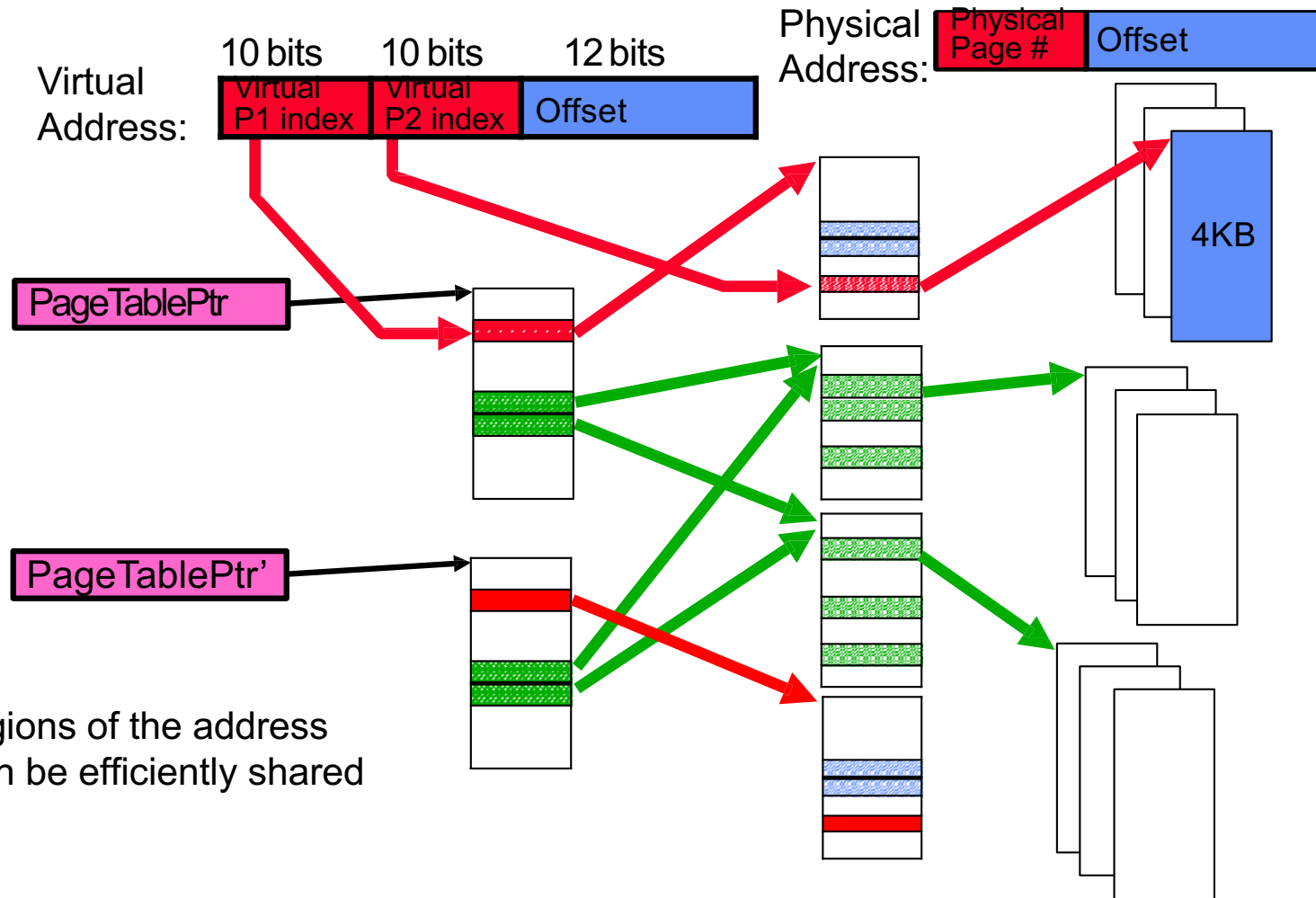
Examples of how to use a PTE

- How do we use the PTE?
 - **Invalid** PTE can imply different things:
 - Region of address space is actually invalid or
 - Page/directory is just somewhere else than memory
 - Validity checked first
 - OS can use other (say) 31 bits for location info
- Usage example: **demand paging**
 - Keep only active pages in memory
 - Place others on disk and mark their PTEs **invalid**

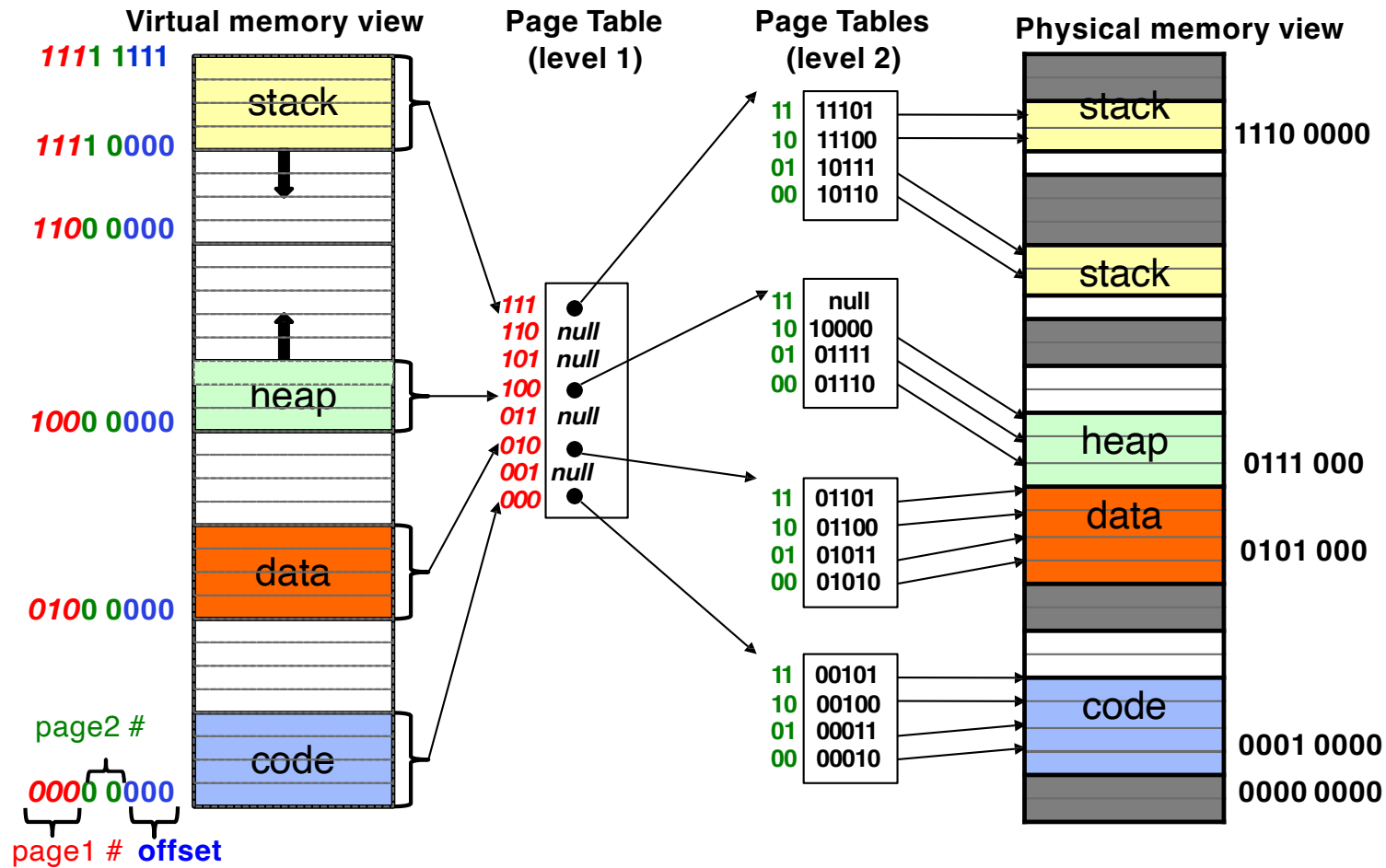
Examples of how to use a PTE

- Usage example: **Copy on Write**
 - UNIX fork gives copy of parent address space to child
 - Address spaces disconnected after child created
 - How to do this cheaply?
 - Make copy of parent's page tables (point at same memory)
 - Mark entries in both sets of page tables as **read-only**
 - Page fault on write creates two copies
- Usage example: **Zero Fill On Demand**
 - New data pages must carry no information (say be zeroed)
 - Mark PTEs as **invalid**; page fault on use gets zeroed page
 - Often, OS creates zeroed pages in background

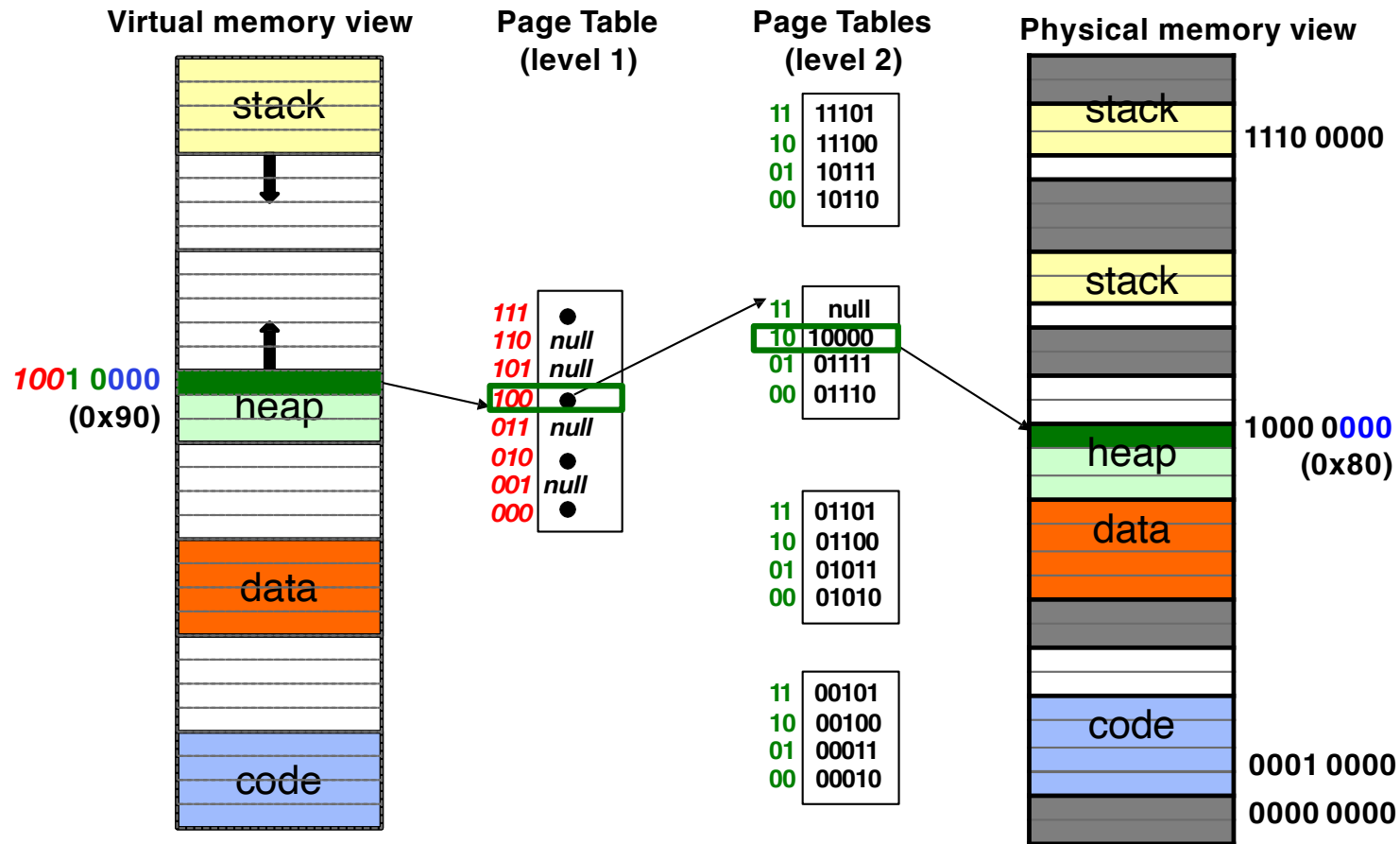
Sharing with multilevel page tables



Summary: two-level paging

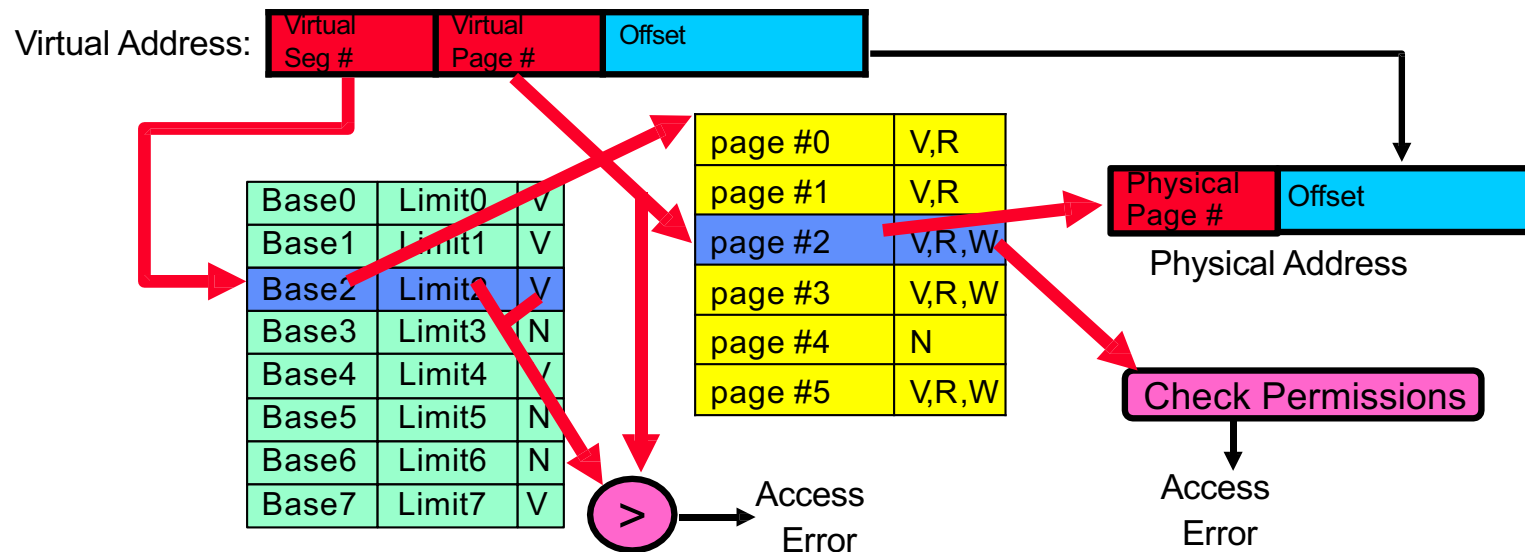


Summary: two-level paging



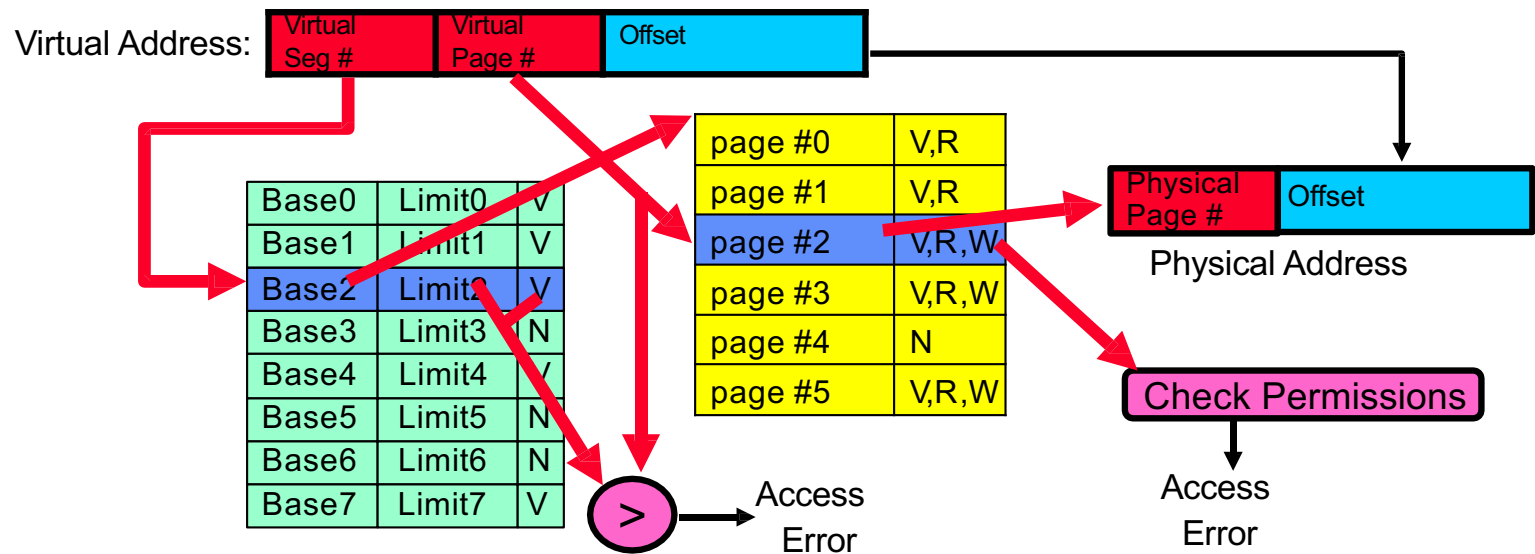
Multi-level translation: segments + pages

- What about a tree of tables?
 - Lowest level page table $\Rightarrow$ memory still allocated with bitmap
 - Higher levels often segmented
- Could have any number of levels. Example (top segment):



Multi-level translation: segments + pages

- What must be saved/restored on context switch?
 - Contents of top-level segment registers (for this example)
 - Pointer to top-level table (page table)



What about sharing (complete segment)?

Process A:

Virtual Seg #	Virtual Page #	Offset
---------------	----------------	--------

Base0	Limit0	V
Base1	Limit1	V
Base2	Limit2	V
Base3	Limit3	N
Base4	Limit4	V
Base5	Limit5	N
Base6	Limit6	N
Base7	Limit7	V

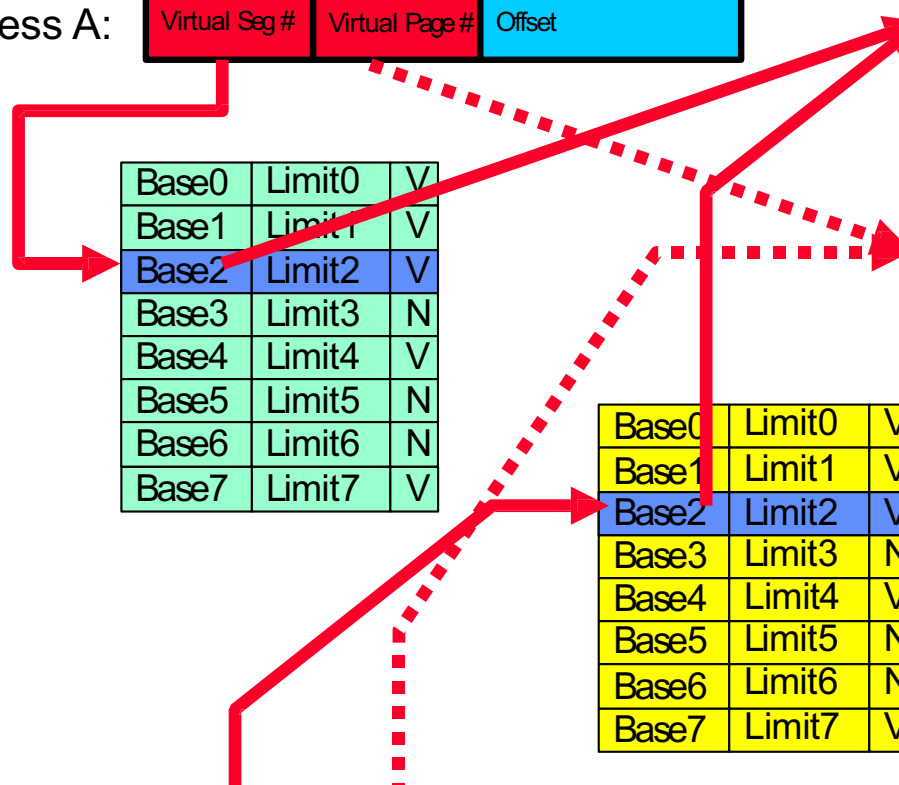
page #0	V,R
page #1	V,R
page #2	V,R,W
page #3	V,R,W
page #4	N
page #5	V,R,W

Shared Segment

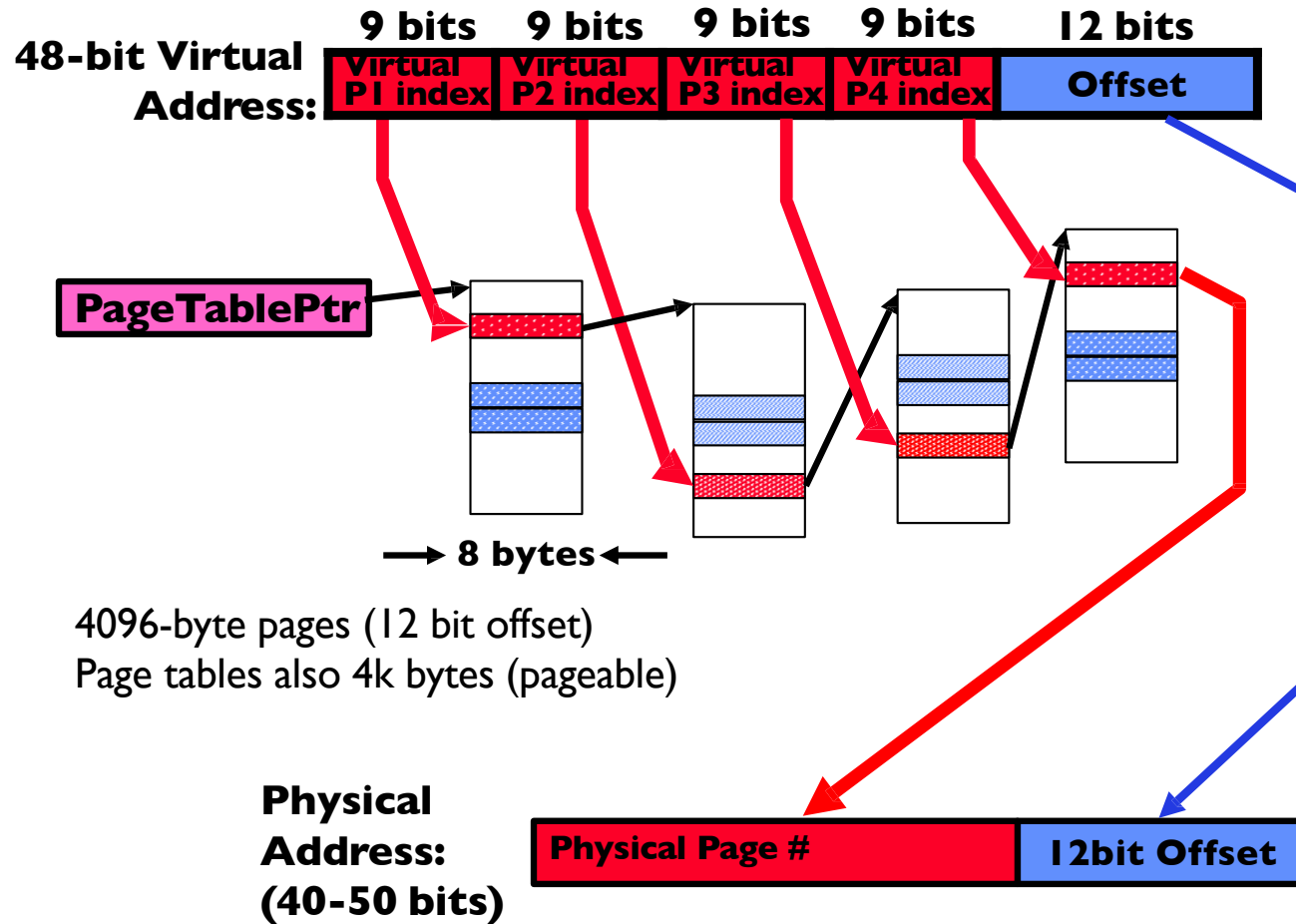
Base0	Limit0	V
Base1	Limit1	V
Base2	Limit2	V
Base3	Limit3	N
Base4	Limit4	V
Base5	Limit5	N
Base6	Limit6	N
Base7	Limit7	V

Process B:

Virtual Seg #	Virtual Page #	Offset
---------------	----------------	--------



X86_64: four-level page table!



Multi-level translation analysis

- Pros:
 - Only need to allocate as many page table entries as we need for application
 - In other words, sparse address spaces are easy
 - Easy memory allocation
 - Easy sharing
 - Share at segment or page level (need additional reference counting)
- Cons:
 - One pointer per page (typically 4K – 16K pages today)
 - Page tables need to be contiguous
 - However, the 10b-10b-12b configuration keeps tables to exactly one page in size
 - Two (or more, if >2 levels) lookups per reference
 - Seems very expensive!

Recall: dual-mode operation

- Can a process modify its own translation tables?
 - **NO!**
 - If it could, could get access to all of physical memory (no protection!)
- To assist with protection, **hardware** provides at least two modes (dual-mode operation):
 - “Kernel” mode (or “supervisor” or “protected”)
 - “User” mode (Normal program mode)
 - Mode set with bit(s) in control register only accessible in Kernel mode
 - Kernel can easily switch to user mode; User program must invoke an exception of some sort to get back to kernel mode (more in moment)

Recall: dual-mode operation

- Note that x86 model actually has more modes:
 - Traditionally, four “rings” representing priority; most OSes use only two:
 - Ring 0 $\Rightarrow$ Kernel mode, Ring 3 $\Rightarrow$ User mode
 - Called “Current Privilege Level” or CPL
 - Newer processors have additional mode for hypervisor (“Ring -1”)
- Certain operations restricted to Kernel mode:
 - Modifying page table base and segment descriptor tables
 - Have to transition into kernel mode before you can change them!
 - Also, all page-table pages must be mapped only in kernel mode

Alternative: Inverted Page Table

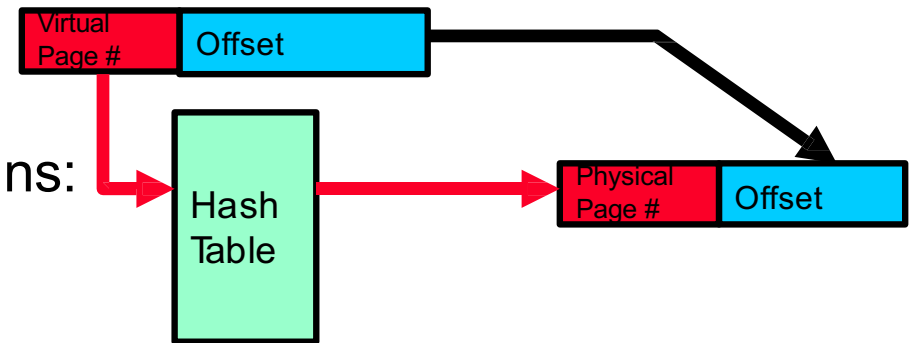
- With all previous examples (“Forward Page Tables”)
 - Size of page table is at least as large as amount of virtual memory allocated to processes
 - Physical memory may be much less
 - Much of process space may be out on disk or not in use

Alternative: Inverted Page Table

- Answer: use a hash table
 - Called an “Inverted Page Table”
 - Size is independent of virtual address space
 - Directly related to amount of physical memory
 - Very attractive option for 64-bit address spaces

- PowerPC, UltraSPARC, IA64

- Cons:
 - Complexity of managing hash chains:
Often in hardware!
 - Poor cache locality of page table



Address translation comparison

	Advantages	Disadvantages
Simple Segmentation	Fast context switching (segment map maintained by CPU)	External fragmentation
Paging (Single-Level)	No external fragmentation Fast and easy allocation	Large table size (~ virtual memory) Internal fragmentation
Paged Segmentation	Table size ~ # of pages in virtual memory Fast and easy allocation	Multiple memory references per page access
Multi-Level Paging		
Inverted Page Table	Table size ~ # of pages in physical memory	Hash function more complex No cache locality of page table

Summary: segmentation and paging

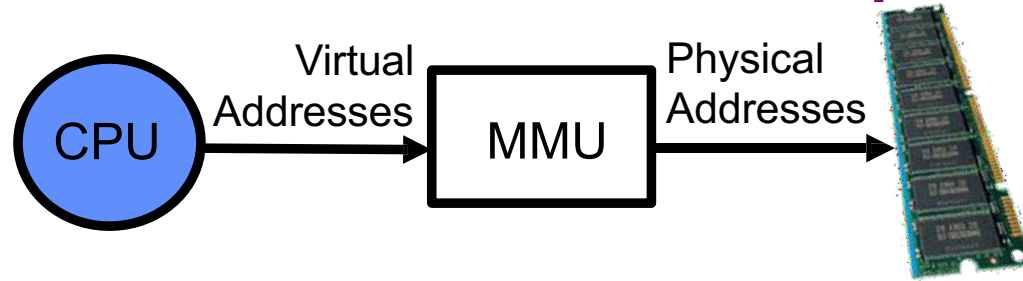
- Segment mapping
 - Segment registers within processor
 - Segment ID associated with each access
 - Often comes from portion of virtual address
 - Can come from bits in instruction instead (x86)
 - Each segment contains base and limit information
 - Offset (rest of address) adjusted by adding base

Summary: segmentation and paging

- Page Tables
 - Memory divided into fixed-sized chunks of memory
 - Virtual page number from virtual address mapped through page table to physical page number
 - Offset of virtual address same as physical address
 - Large page tables can be placed into virtual memory
- Multi-level tables
 - Virtual address mapped to series of tables
 - Permit sparse population of address space

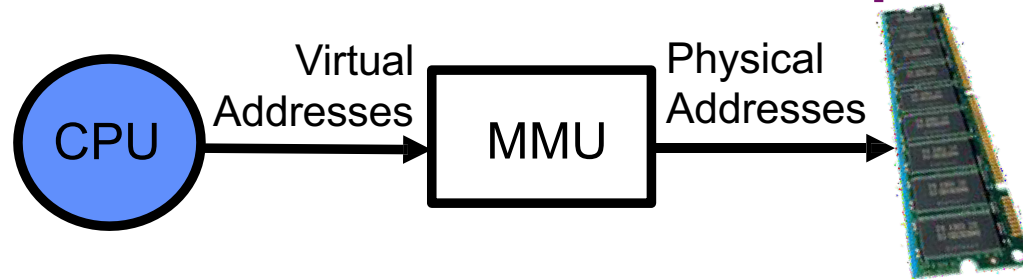
Caching

How is the translation accomplished?



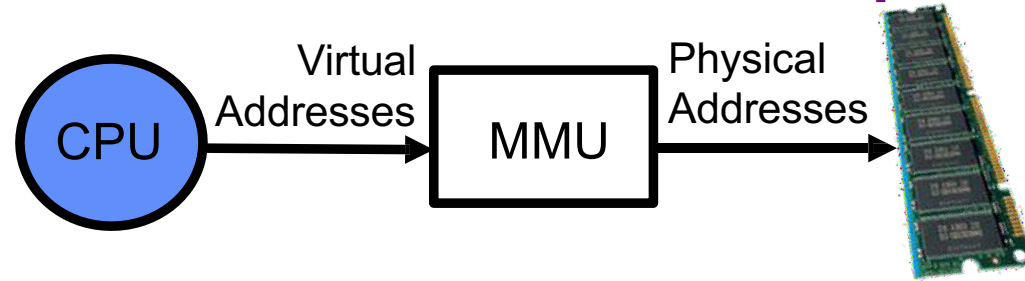
- The MMU must attempt to translate virtual address to physical address on:
 - *Every instruction fetch, every load, every store*
 - Generate a “Page Fault” (Trap) if it encounters invalid PTE
 - Fault handler will decide what to do (more on this next lecture)

How is the translation accomplished?



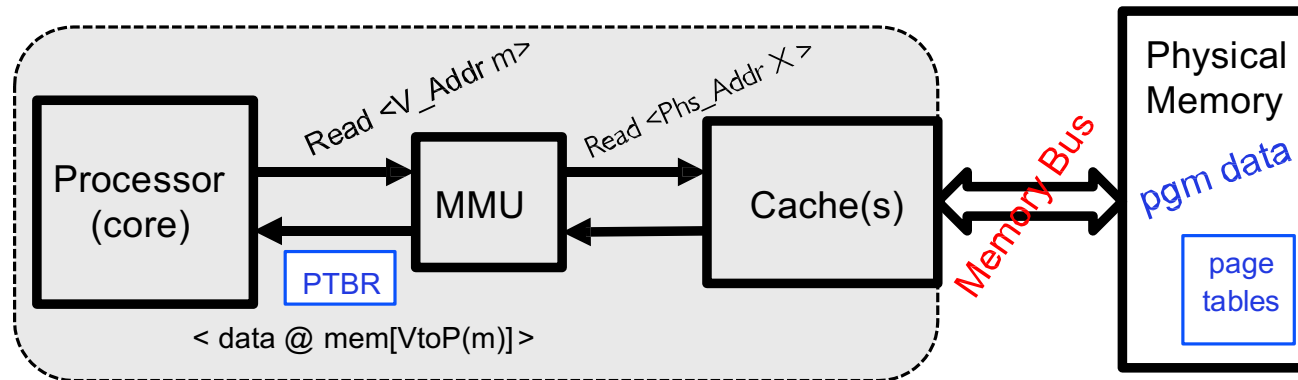
- What does the MMU need to do to translate an address?
 - 1-level Page Table
 - Read PTE from memory, check valid, merge address
 - Set “accessed” bit in PTE, Set “dirty bit” on write
 - 2-level Page Table
 - Read and check first level
 - Read, check, and update PTE at second level
 - N-level Page Table ...

How is the translation accomplished?



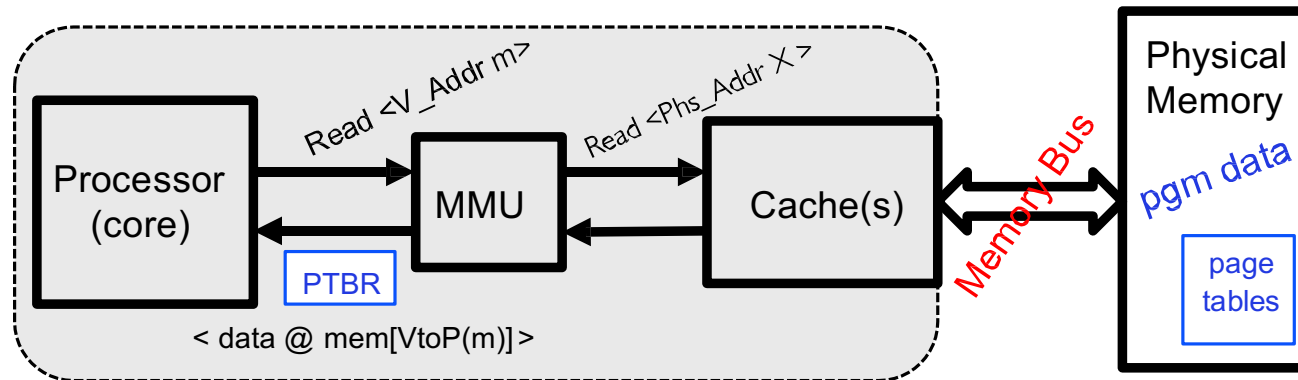
- MMU does *page table Tree Traversal* to translate each address
 - Turns a potentially fast memory access into a slow multi-access table traversal...
 - Need CACHING!

Where and what is the MMU ?



- The processor requests READ Virtual-Address to memory system
 - Through the MMU to the cache (to the memory)
- Some time later, the memory system responds with the data stored at the physical address (resulting from virtual → physical) translation
 - Fast on a cache hit, slow on a miss

Where and what is the MMU ?



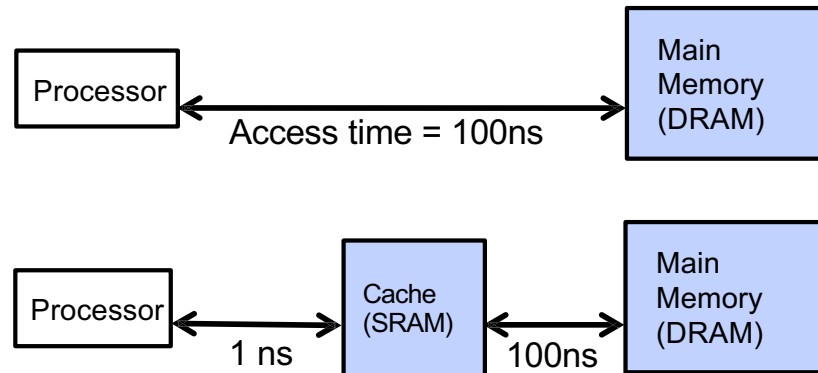
- So what is the MMU doing?
- On every reference (instruction fetch, load, store) read (multiple levels of) page table entries to get physical frame or FAULT
 - Through the caches to the memory
 - Then read/write the physical location

Caching concept

- **Cache**: a repository for copies that can be accessed more quickly than the original
 - Make frequent case fast and infrequent case less dominant
- Caching underlies many techniques used today to make computers fast
 - Can cache: memory locations, address translations, pages, file blocks, file names, network routes, etc...
- Only good if:
 - Frequent case frequent enough and
 - Infrequent case not too expensive
- Many important OS concepts boil down to caching! We cache:
 - Pages, Files, Virtual Memory Translations, IP Addresses...

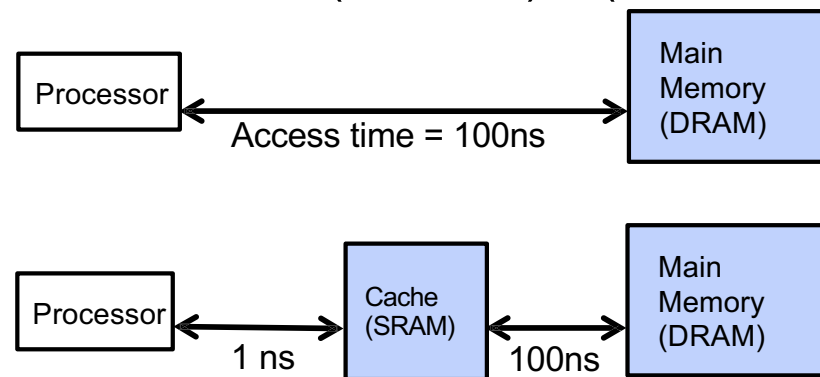
Caching concept

- Hardware Caching is the key to memory system performance for CPUs:

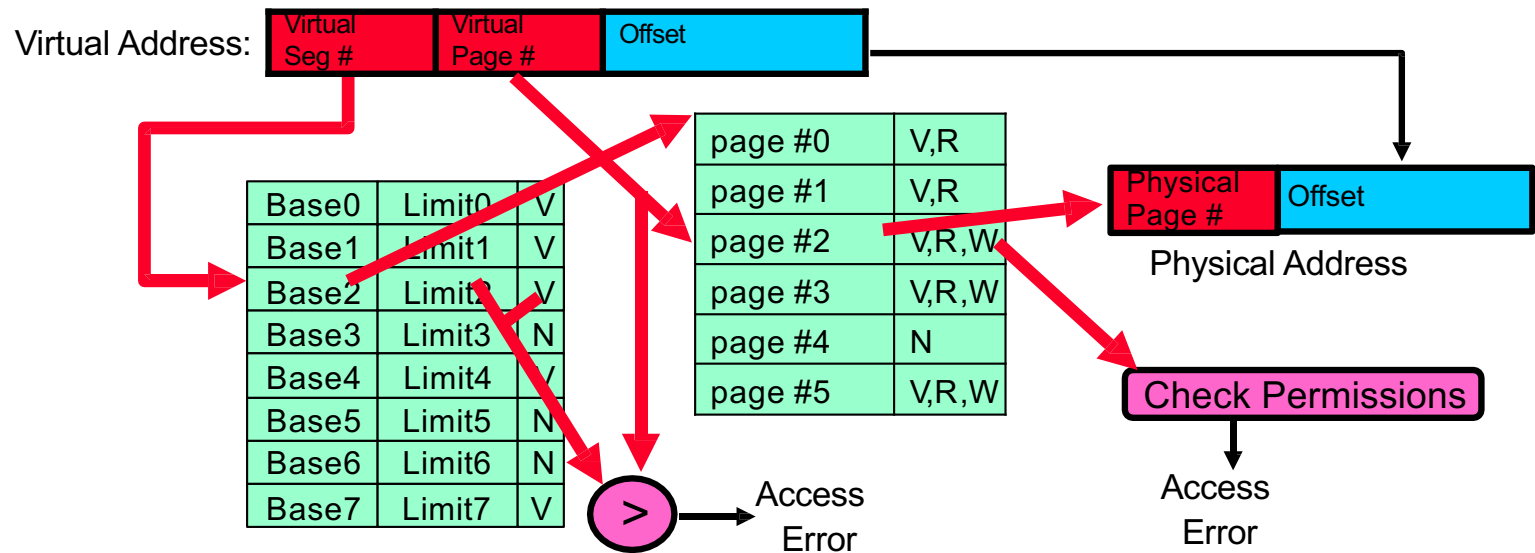


Caching concept

- Average Memory Access Time (AMAT)
$$= (\text{Hit Rate} \times \text{HitTime}) + (\text{Miss Rate} \times \text{MissTime})$$
- Where:
 - $\text{HitRate} + \text{MissRate} = 1$
 - $\text{MissTime} = \text{HitTime} + \text{MissPenalty}$
- Examples:
 - $\text{HitRate} = 90\% \Rightarrow \text{AMAT} = (0.9 \times 1) + (0.1 \times 101) = 11 \text{ ns}$
 - $\text{HitRate} = 99\% \Rightarrow \text{AMAT} = (0.99 \times 1) + (0.01 \times 101) = 2.01 \text{ ns}$

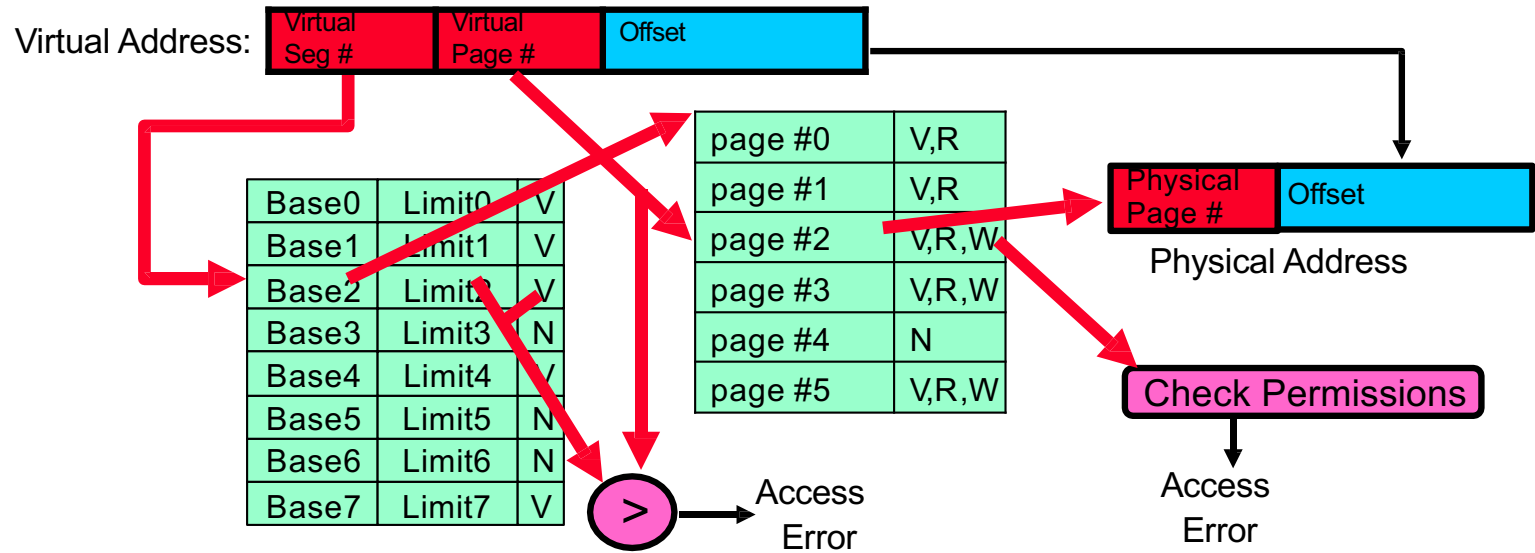


Another major reason to deal with caching



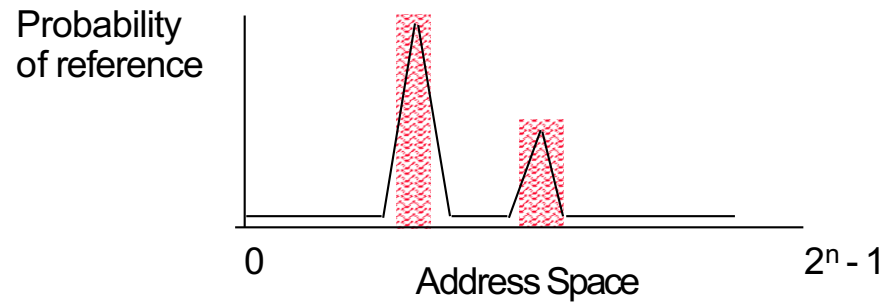
- Cannot afford to translate on every access
 - At least three DRAM accesses per actual DRAM access
 - Or: perhaps I/O if page table partially on disk!
- Even worse: What if we are using caching to make memory access faster than DRAM access?

Another major reason to deal with caching

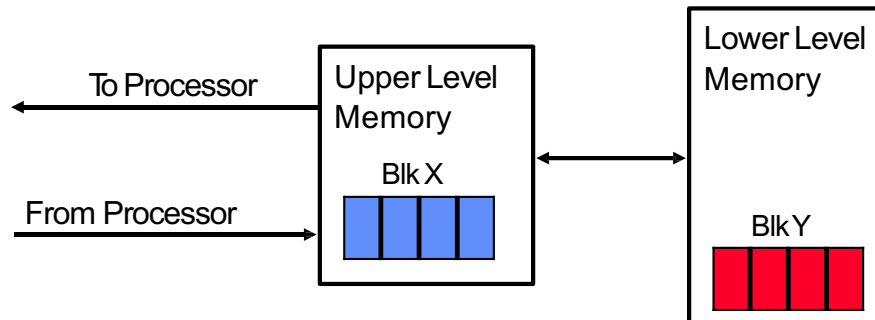


- Solution? Cache translations!
 - Translation Cache: TLB (“Translation Lookaside Buffer”)

Why does caching help? Locality!

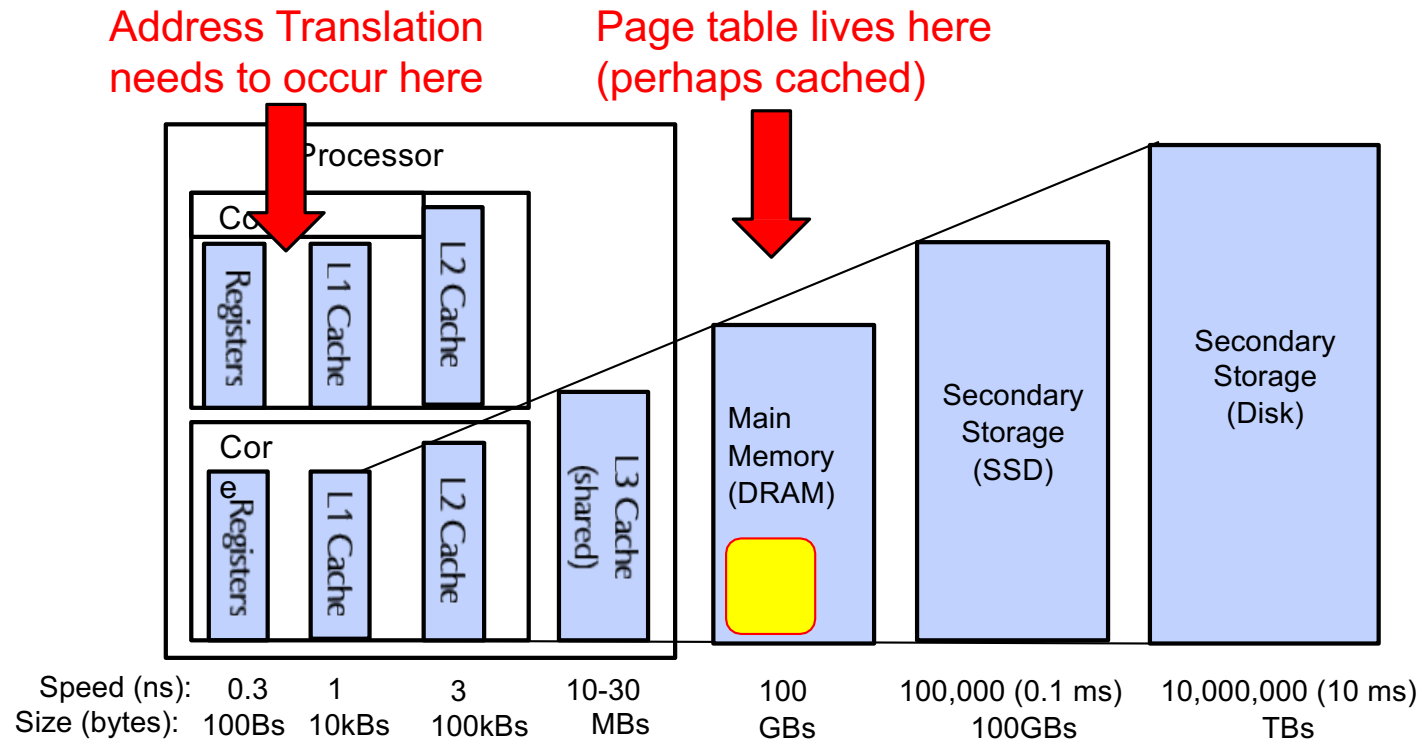


- **Temporal Locality** (Locality in Time):
 - Keep recently accessed data items closer to processor
- **Spatial Locality** (Locality in Space):
 - Move contiguous blocks to the upper levels



Memory hierarchy

- Caching: take advantage of the principle of locality to:
 - Present the illusion of having as much memory as in the cheapest technology
 - Provide average speed similar to that offered by the fastest technology



Dealing with hierarchy

- Used to compute access time probabilistically:

$$AMAT = \text{Hit Rate}_{L1} \times \text{Hit Time}_{L1} + \text{Miss Rate}_{L1} \times \text{Miss Time}_{L1}$$

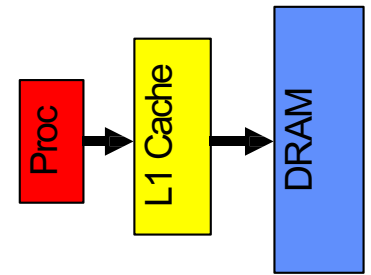
$$\text{Hit Rate}_{L1} + \text{Miss Rate}_{L1} = 1$$

$$\text{Hit Time}_{L1} = \text{Time to get value from L1 cache.}$$

$$\text{Miss Time}_{L1} = \text{Hit Time}_{L1} + \text{Miss Penalty}_{L1}$$

$$\text{Miss Penalty}_{L1} = \text{AVG Time to get value from lower level (DRAM)}$$

$$\text{So, } AMAT = \text{Hit Time}_{L1} + \text{Miss Rate}_{L1} \times \text{Miss Penalty}_{L1}$$



- What about more levels of hierarchy?

$$AMAT = \text{Hit Time}_{L1} + \text{Miss Rate}_{L1} \times \text{Miss Penalty}_{L1}$$

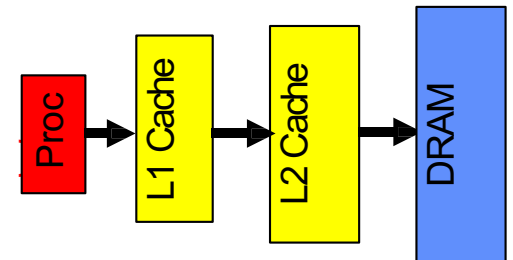
$$\text{Miss Penalty}_{L1} = \text{AVG time to get value from lower level (L2)}$$

$$= \text{Hit Time}_{L2} + \text{Miss Rate}_{L2} \times \text{Miss Penalty}_{L2}$$

$$\text{Miss Penalty}_{L2} = \text{Average Time to fetch from below L2 (DRAM)}$$

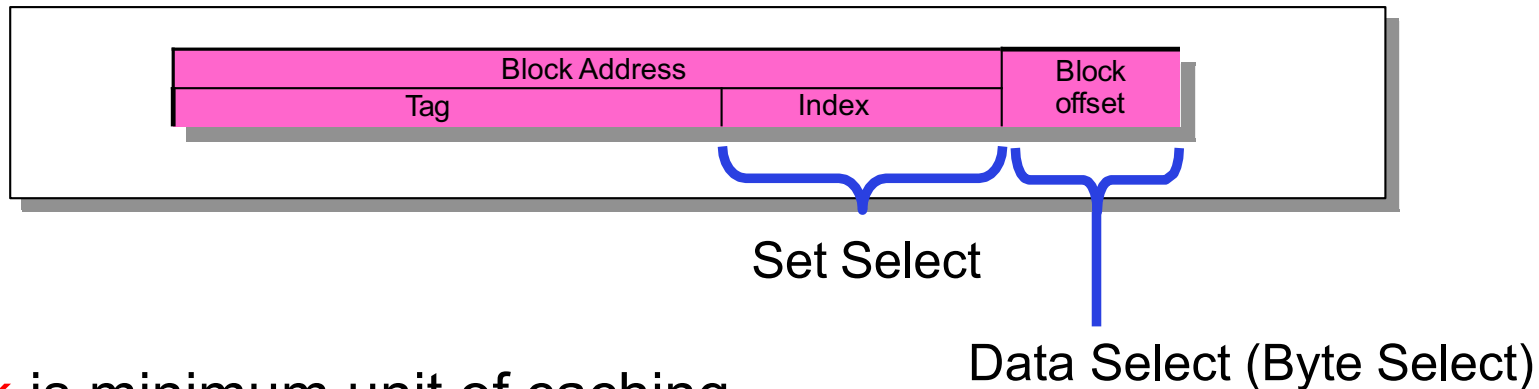
$$AMAT = \text{Hit Time}_{L1} +$$

$$\text{Miss Rate}_{L1} \times (\text{Hit Time}_{L2} + \text{Miss Rate}_{L2} \times \text{Miss Penalty}_{L2})$$



- And so on ... (can do this recursively for more levels!)

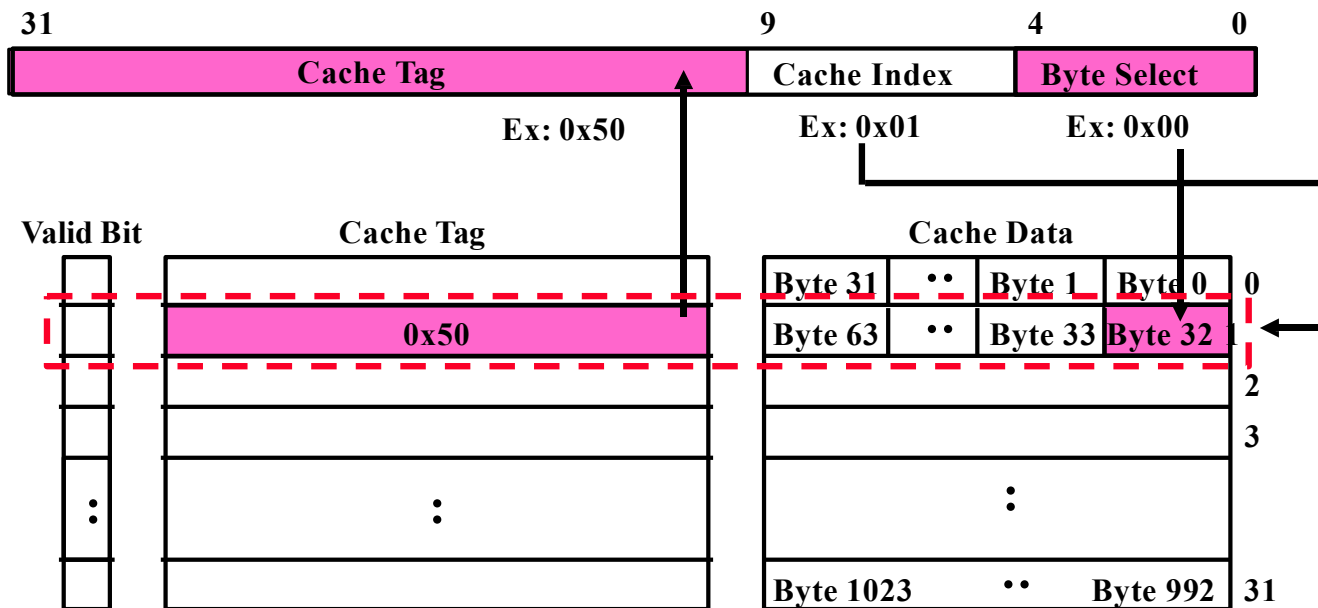
How is a Block found in a Cache?



- **Block** is minimum unit of caching
 - Data select field used to select data within block
 - Many caching applications don't have data select field
- **Index** Used to Lookup Candidates in Cache
 - Index identifies the set
- **Tag** used to identify actual copy
 - If no candidates match, then declare cache miss

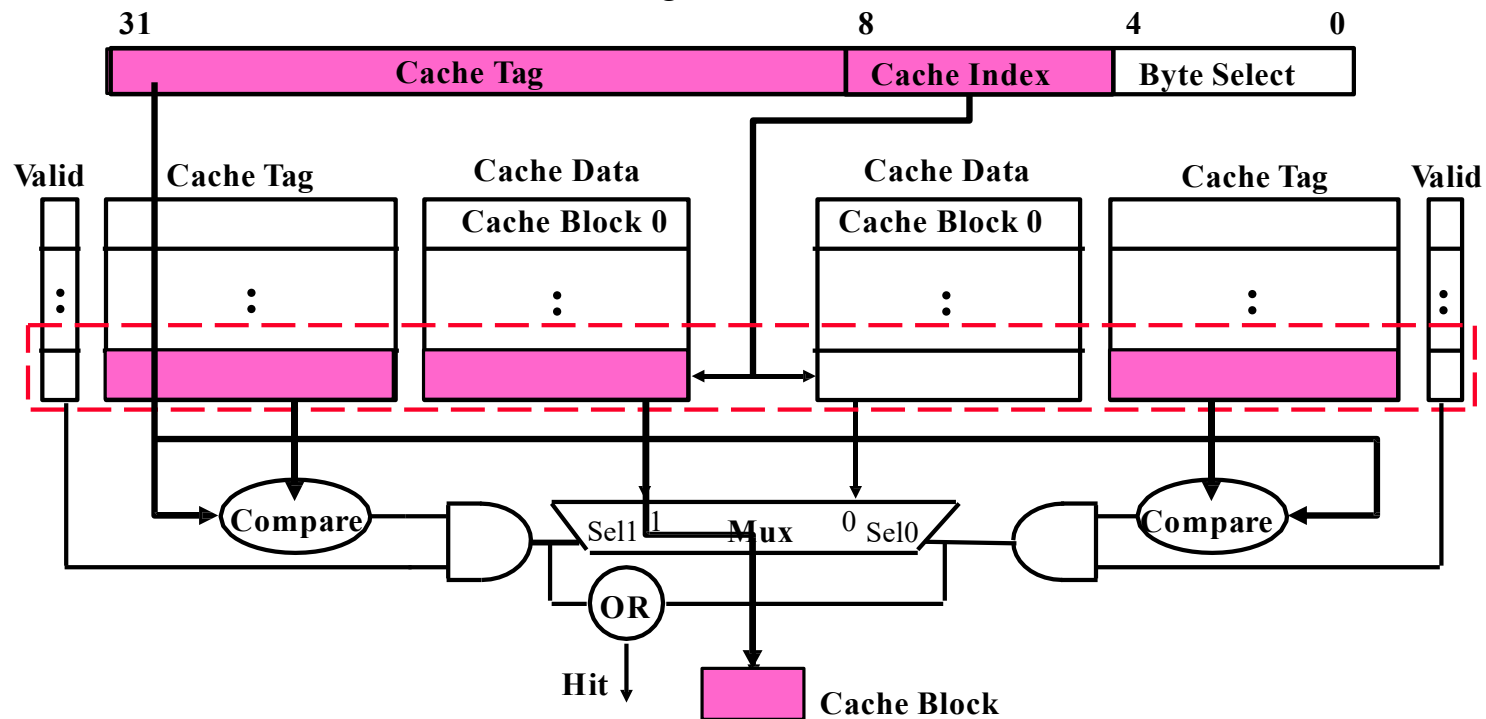
Review: Direct Mapped Cache

- **Direct Mapped 2^N byte cache:**
 - The uppermost $(32 - N)$ bits are always the Cache Tag
 - The lowest M bits are the Byte Select (Block Size = 2^M)
- Example: 1 KB Direct Mapped Cache with 32 B Blocks
 - Index chooses potential block
 - Tag checked to verify block
 - Byte select chooses byte within block



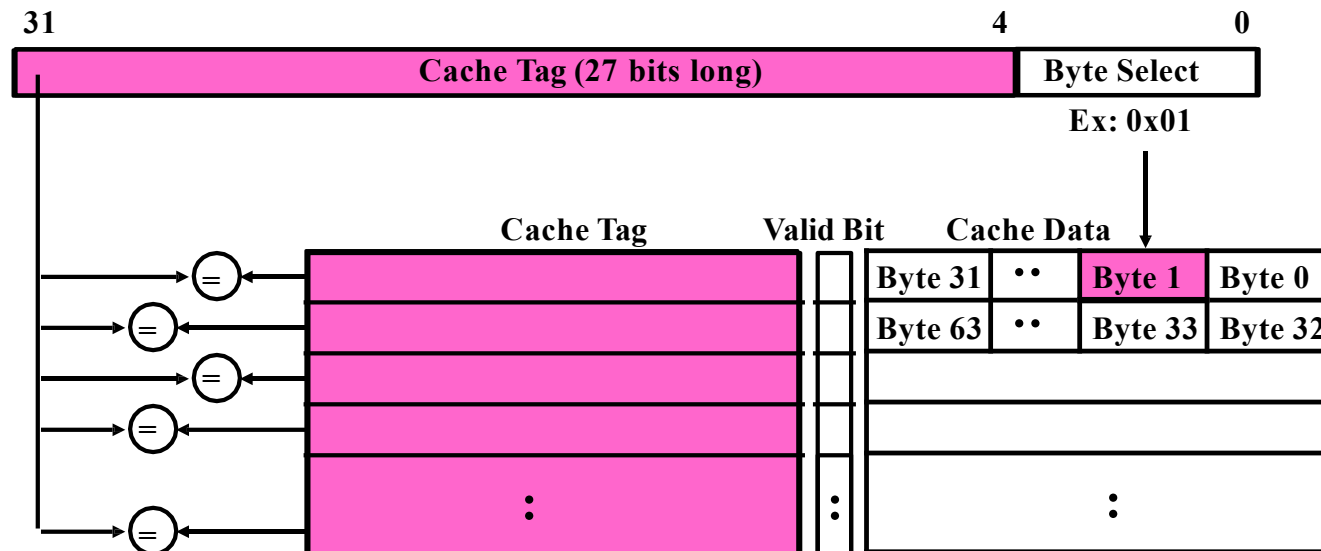
Review: Set Associative Cache

- **N-way set associative**: N entries per Cache Index
 - N direct mapped caches operates in parallel
- Example: Two-way set associative cache
 - Cache Index selects a “set” from the cache
 - Two tags in the set are compared to input in parallel
 - Data is selected based on the tag result



Review: Fully Associative Cache

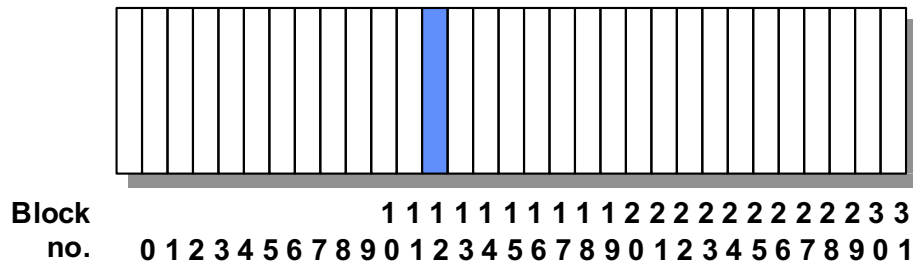
- **Fully Associative**: Every block can hold any line
 - Address does not include a cache index
 - Compare Cache Tags of all Cache Entries in Parallel
- Example: Block Size=32B blocks
 - We need N 27-bit comparators
 - Still have byte select to choose from within block



Where does a Block get placed in a Cache?

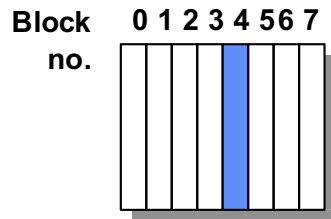
- Example: Block 12 placed in 8 block cache

32-Block Address Space:



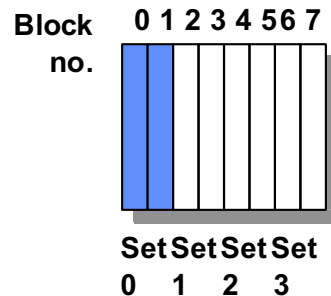
Direct mapped:

**block 12 can go
only into block 4
(12 mod 8)**



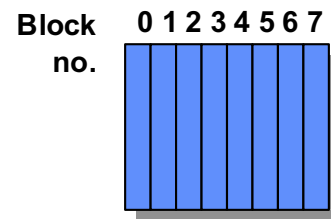
Set associative:

**block 12 can go
anywhere in set 0
(12 mod 4)**



Fully associative:

**block 12 can go
anywhere**



Which block should be replaced on a miss?

- Easy for Direct Mapped: only one possibility
- Set Associative or Fully Associative:
 - Random
 - LRU (Least Recently Used)
- Miss rates for a workload:

Size	2-way		4-way		8-way	
	LRU	Random	LRU	Random	LRU	Random
16 KB	5.2%	5.7%	4.7%	5.3%	4.4%	5.0%
64 KB	1.9%	2.0%	1.5%	1.7%	1.4%	1.5%
256 KB	1.15%	1.17%	1.13%	1.13%	1.12%	1.12%

Review: what happens on a write?

- **Write through:** the information is written to both the block in the cache and to the block in the lower-level memory
- **Write back:** the information is written only to the block in the cache
 - Modified cache block is written to main memory only when it is replaced
 - Question is block clean or dirty?

Pros and cons

- Write through:
 - PRO:
 - Read misses cannot result in writes
 - CON:
 - Processor held up on writes unless writes buffered
- Write back:
 - PRO:
 - repeated writes not sent to DRAM
 - processor not held up on writes
 - CON:
 - More complex
 - Read miss may require writeback of dirty data

Sources of cache misses

- **Compulsory** (cold start or process migration, first reference): first access to a block
 - “Cold” fact of life: not a whole lot you can do about it unless you prefetch
 - Solution: prefetch values before use
 - Note: if you run “billions” of instruction, compulsory misses are insignificant

Sources of cache misses

- Capacity:

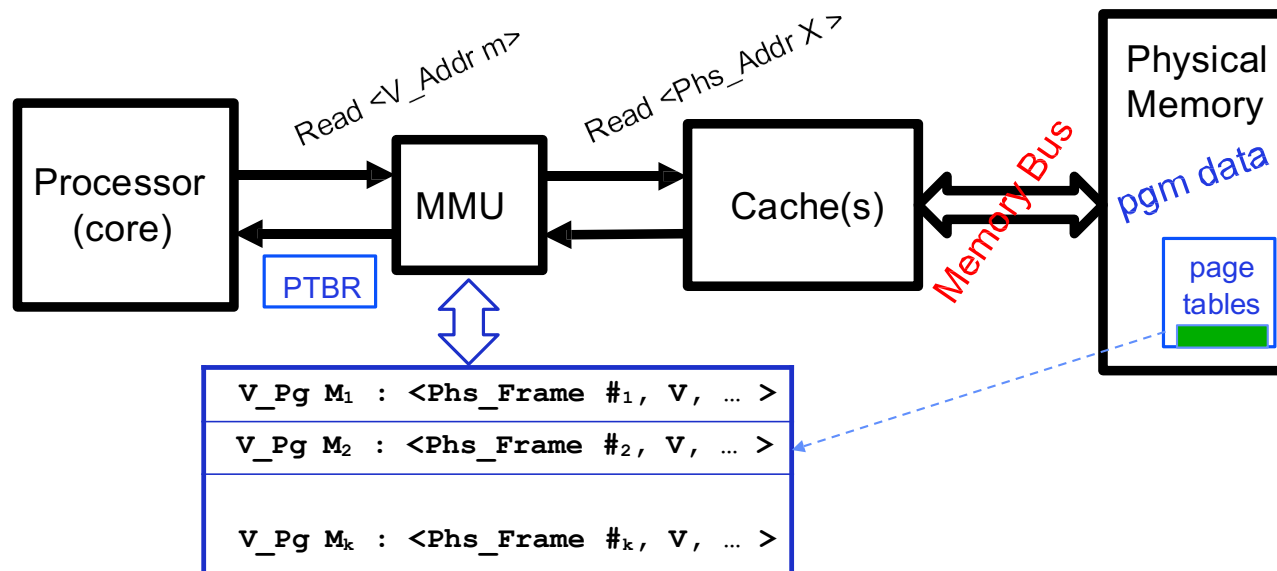
- Cache cannot contain all blocks access by the program
- Solution 1: increase cache size
- “Solution 2”: Change (e.g. reduce) associativity to focus misses in a few places?!
 - Consider fully-associative cache of size n : access pattern $0, 1, \dots, n-1, n, 0, 1, \dots$
 - Contrast with direct mapped of size n : Fewer misses!

Sources of cache misses

- **Conflict (collision):**
 - Multiple memory locations mapped to the same cache location
 - Solution 1: increase cache size
 - Solution 2: increase associativity
- **Coherence (Invalidation):**
 - other process (e.g., I/O) updates memory

How do we make address translation fast?

- Cache results of recent translations!
 - Different from a traditional cache
 - Cache Page Table Entries using Virtual Page # as the key



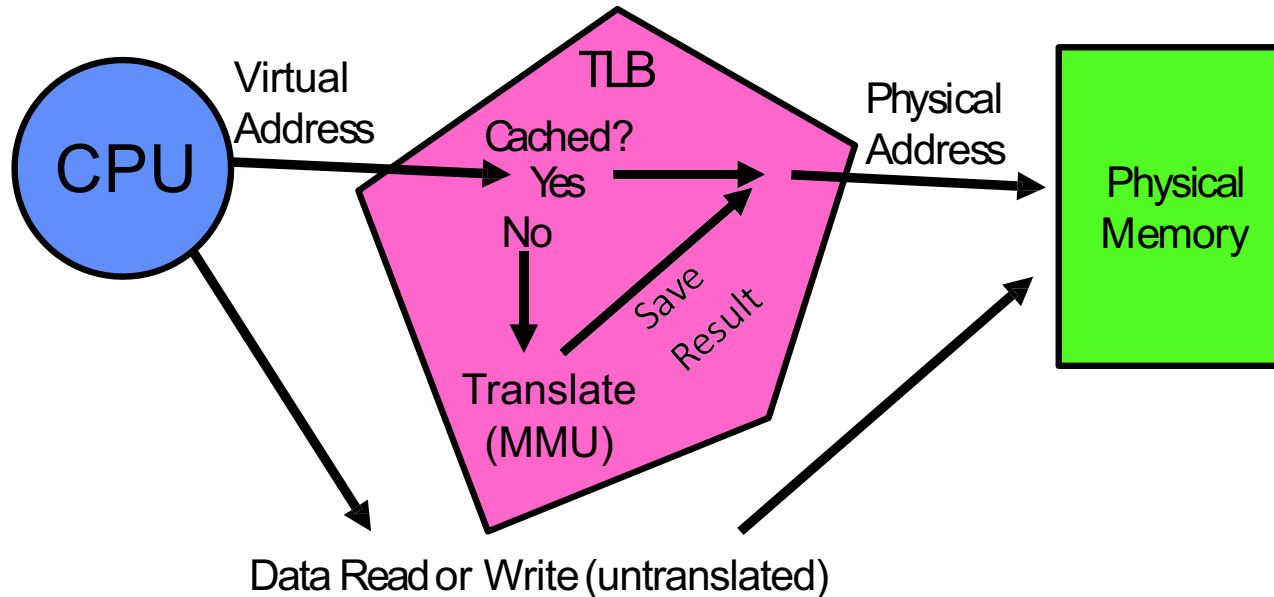
Translation Look-Aside Buffer (TLB)

- Record recent Virtual Page # to Physical Frame # translation
- If present, have the physical address without reading any page tables!!!
 - Even if the translation involved multiple levels
 - Caches the end-to-end result
- Was invented by Sir Maurice Wilkes – *prior to caches*
 - When you come up with a new concept, you get to name it!
 - People realized “if it’s good for page tables, why not the rest of the data in memory?”
- On a *TLB miss*, the page tables may be cached, so only go to memory when both miss



Maurice Wilkes
Turing Award: 1967
(1913 – 2010)

Caching applied to address translation



- Question is one of page locality: does it exist?
 - Instruction accesses spend a lot of time on same page (accesses are sequential)
 - Stack accesses have definite locality of reference
 - Data accesses have less page locality, but still some...
- Can we have a TLB hierarchy?
 - Sure: multiple levels at different sizes/speeds

Summary

- Major Categories of Cache Misses:
 - **Compulsory Misses:** sad facts of life. Example: cold start misses.
 - **Conflict Misses:** increase cache size and/or associativity
 - **Capacity Misses:** increase cache size
 - **Coherence Misses:** Caused by external processors or I/O devices
- Cache Organizations:
 - Direct Mapped: single block per set
 - Set associative: more than one block per set
 - Fully associative: all entries equivalent